

API INTEGRATION & JSON HANDLING

User Information Search & Filter Application

Name: Bammidi Rakesh

Roll Number: 23981A04Q9

Department: ECE

College: Raghu Engineering College

Date: 8 September 2026

1. Introduction

This project demonstrates Python communication with an external REST API, JSON parsing, search and filtering, and API error handling. The public JSONPlaceholder Users API is used for testing without an API key.

2. Objectives

- Fetch data using the Requests library.
- Parse JSON responses.
- Search users by name.
- Filter users by city.
- Refresh API data.
- Handle API errors gracefully.

3. Requirements Mapping

Requirement	Implementation
Fetch data using Requests	<code>requests.get(API_URL, timeout=10)</code>
Parse JSON	<code>response.json()</code>
Filtering/search	Name search and city filtering
Handle API errors	Timeout, HTTPError, RequestException and ValueError

4. Tools and Technologies

- Python 3
- Requests library
- JSON
- Visual Studio Code
- JSONPlaceholder REST API

5. API Endpoint

<https://jsonplaceholder.typicode.com/users>

6. Program Features

- Display all users
- Search user by name
- Filter users by city
- Refresh API data
- Exception handling
- Menu-driven interface

7. JSON Handling

`response.json()` converts the JSON response into Python objects. Nested address and city fields are used for filtering.

8. Error Handling

The program catches timeout, HTTP, connection/request, and invalid JSON errors so the application can continue safely.

9. Program Flow

- Start the application.
- Send a GET request.
- Check the HTTP status.
- Parse JSON.
- Display the main menu.
- Perform display, search, filter, refresh, or exit.

10. Complete Source Code

```
import requests

# Public API URL
API_URL = "https://jsonplaceholder.typicode.com/users"

def fetch_users():
    try:
        response = requests.get(API_URL, timeout=10)
        response.raise_for_status()
        users = response.json()
        print("\nAPI data fetched successfully!")
        print("Total users:", len(users))
        return users
    except requests.exceptions.Timeout:
        print("\nError: API request timed out.")
        return []
    except requests.exceptions.HTTPError as error:
        print("\nHTTP Error:", error)
        return []
    except requests.exceptions.RequestException as error:
        print("\nAPI Connection Error:", error)
        return []
    except ValueError:
        print("\nError: Invalid JSON response.")
        return []

def display_users(users):
    if not users:
        print("\nNo user data available.")
        return
    print("\n----- USER LIST -----")
    for user in users:
        print(f"ID: {user['id']} | Name: {user['name']} | Username: {user['username']} | City: {user['address']['city']}")
    print("-----")

def search_user(users):
    search_name = input("\nEnter user name to search: ").strip().lower()
    found = False
    for user in users:
        if search_name in user["name"].lower():
            print("\nUser found!")
            print("ID      :", user["id"])
            print("Name    :", user["name"])
            print("Username:", user["username"])
            print("Email   :", user["email"])
            print("Phone   :", user["phone"])
            print("City    :", user["address"]["city"])
            found = True
    if not found:
        print("\nNo user found with that name.")

def filter_by_city(users):
    city = input("\nEnter city to filter: ").strip().lower()
    filtered_users = [u for u in users if city in u["address"]["city"].lower()]
    if filtered_users:
        print("\nUsers from", city.title(), ":")
        for user in filtered_users:
            print(f"ID: {user['id']} | Name: {user['name']} | City: {user['address']['city']}")
        print("Total matching users:", len(filtered_users))
    else:
        print("\nNo users found in that city.")

print("=====")
print("      USER INFORMATION API")
print("=====")
users = fetch_users()

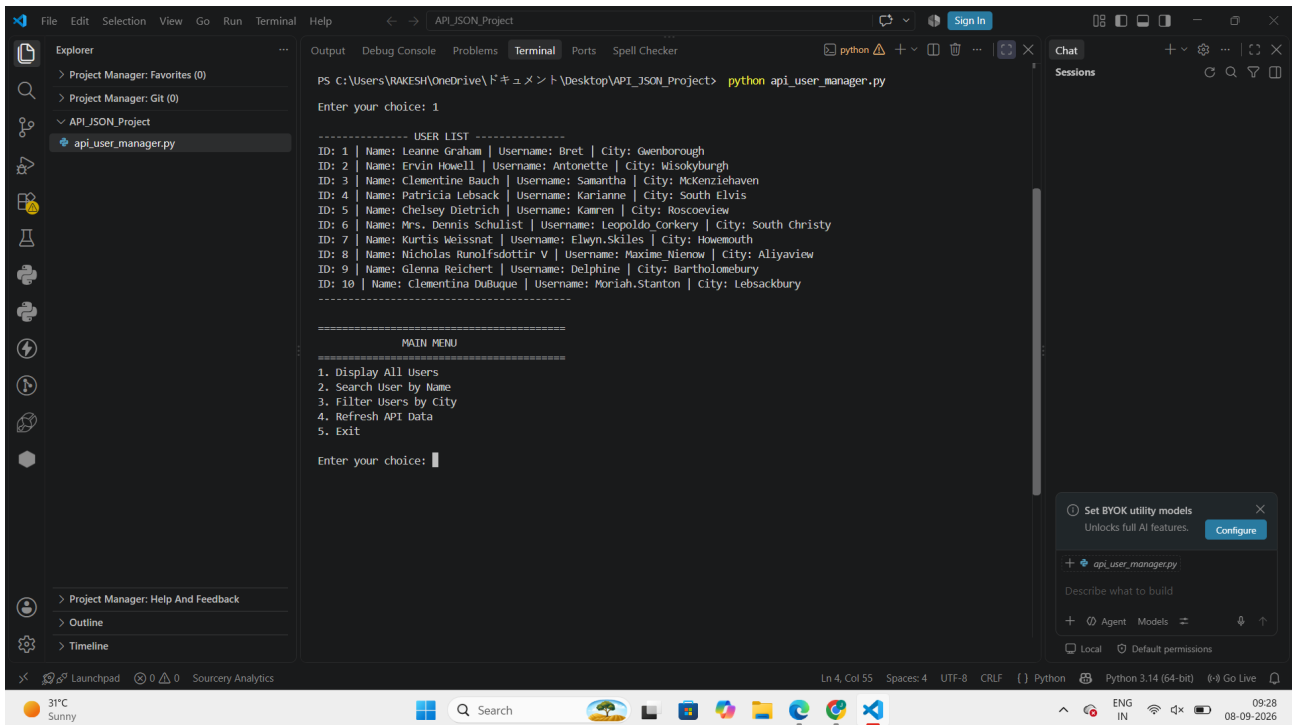
while True:
    print("\n=====")
    print("      MAIN MENU")
    print("=====")
    print("1. Display All Users")
    print("2. Search User by Name")
    print("3. Filter Users by City")
    print("4. Refresh API Data")
    print("5. Exit")
    choice = input("\nEnter your choice: ")

    if choice == "1":
```

```
        display_users(users)
    elif choice == "2":
        search_user(users) if users else print("\nNo API data available.")
    elif choice == "3":
        filter_by_city(users) if users else print("\nNo API data available.")
    elif choice == "4":
        users = fetch_users()
    elif choice == "5":
        print("\nThank you for using User Information API.")
        print("Program exited successfully.")
        break
    else:
        print("\nInvalid choice. Please enter 1 to 5.")
```

11. Output Screenshots and Explanations

1. API Data Fetching



```
PS C:\Users\RAKESH\OneDrive\ドキュメント\Desktop\API_JSON_Project> python api_user_manager.py

Enter your choice: 1

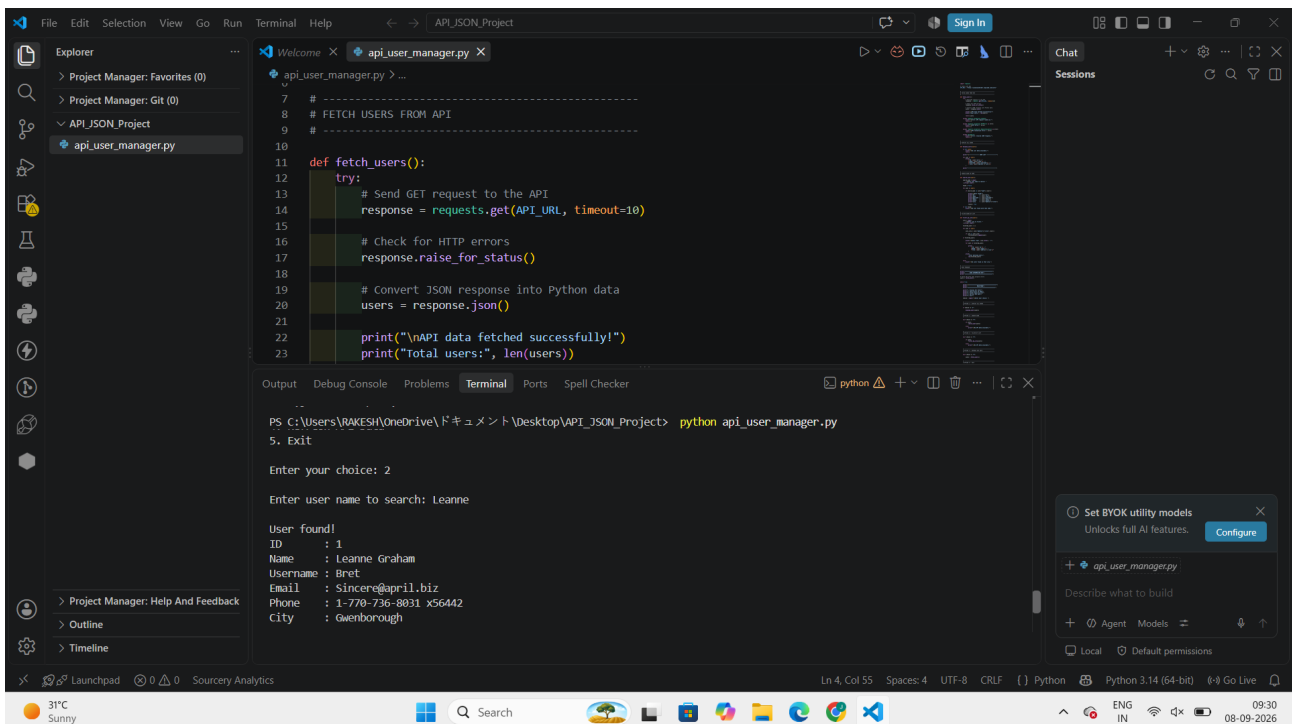
----- USER LIST -----
ID: 1 | Name: Leanne Graham | Username: Bret | City: Gwenborough
ID: 2 | Name: Ervin Howell | Username: Antonette | City: Wisokyburgh
ID: 3 | Name: Clementine Bauch | Username: Samantha | City: McKenziehaven
ID: 4 | Name: Patricia Lebsack | Username: Karianne | City: South Elvis
ID: 5 | Name: Chelsey Dietrich | Username: Kamren | City: Roscoeview
ID: 6 | Name: Mrs. Dennis Schulist | Username: Leopoldo Corkery | City: South Christy
ID: 7 | Name: Kurtis Weissnat | Username: Elwyn Skiles | City: Howemouth
ID: 8 | Name: Nicholas Runolfsdottir V | Username: Maxime Nienow | City: Aliyaview
ID: 9 | Name: Glenna Reichert | Username: Delphine | City: Bartholomebury
ID: 10 | Name: Clementina DuBuque | Username: Moriah Stanton | City: Lebsackbury
-----

----- MAIN MENU -----
1. Display All Users
2. Search User by Name
3. Filter Users by City
4. Refresh API Data
5. Exit

Enter your choice: 
```

The application sends a GET request using Requests, parses the JSON response, and displays 10 user records.

2. Search User by Name



```
def fetch_users():
    try:
        # Send GET request to the API
        response = requests.get(API_URL, timeout=10)

        # Check for HTTP errors
        response.raise_for_status()

        # Convert JSON response into Python data
        users = response.json()

        print("\nAPI data fetched successfully!")
        print("Total users:", len(users))

    except requests.exceptions.RequestException as e:
        print(f"Error: {e}")

# Search user by name
def search_user_by_name():
    name = input("Enter user name to search: ")

    for user in users:
        if user['Name'] == name:
            print("User found!")
            print(f"ID: {user['ID']}")
            print(f>Name: {user['Name']}")
            print(f>Username: {user['Username']}")
            print(f>Email: {user['Email']}")
            print(f>Phone: {user['Phone']}")
            print(f>City: {user['City']}")
            return
    print("User not found.")

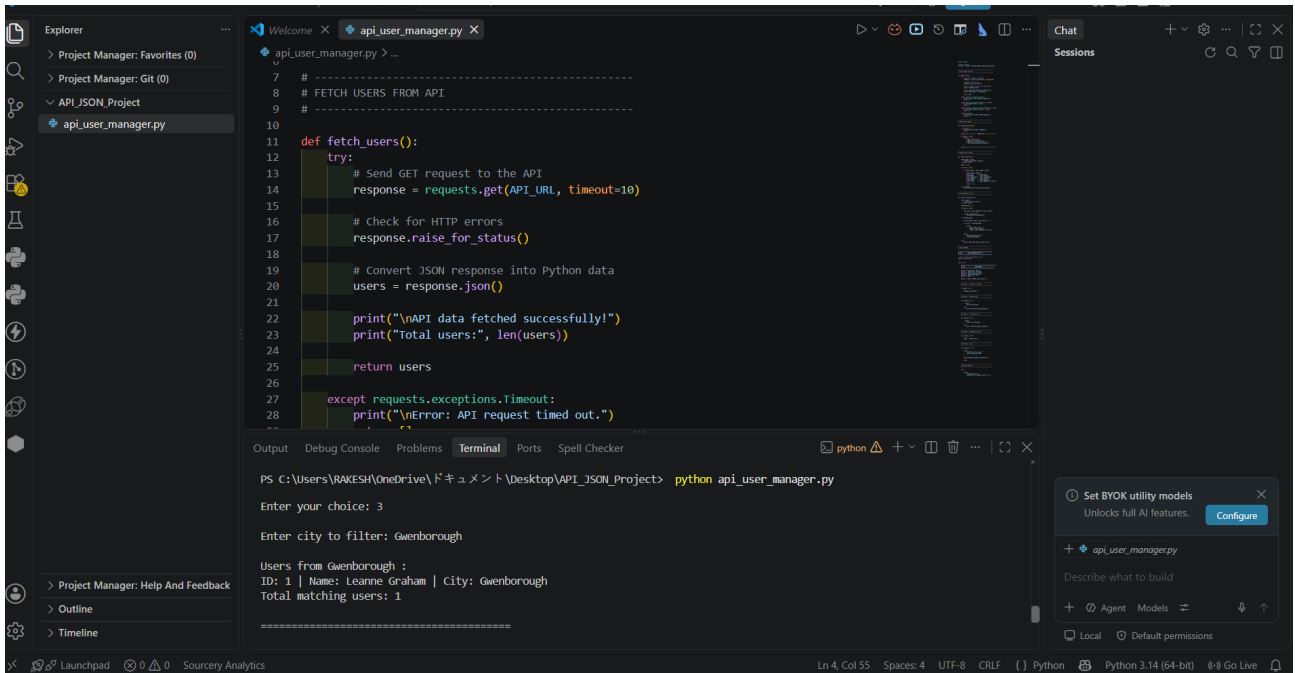
# Main menu
def main():
    while True:
        print("\n----- MAIN MENU -----")
        print("1. Display All Users")
        print("2. Search User by Name")
        print("3. Filter Users by City")
        print("4. Refresh API Data")
        print("5. Exit")

        choice = input("Enter your choice: ")

        if choice == '1':
            fetch_users()
        elif choice == '2':
            search_user_by_name()
        elif choice == '3':
            filter_users_by_city()
        elif choice == '4':
            refresh_api_data()
        elif choice == '5':
            exit()
        else:
            print("Invalid choice. Please try again.")
```

The program searches the parsed JSON data for a matching name and displays the selected user's details.

3. Filter Users by City



The screenshot shows the VS Code editor with the file `api_user_manager.py` open. The code defines a `fetch_users()` function that sends a GET request to an API, checks for errors, and returns the JSON response. The terminal output shows the command `python api_user_manager.py` being executed, followed by the prompt `Enter your choice: 3` and `Enter city to filter: Gwenborough`. The output displays the users from Gwenborough: `ID: 1 | Name: Leanne Graham | City: Gwenborough` and `Total matching users: 1`.

```
7 # -----
8 # FETCH USERS FROM API
9 # -----
10
11 def fetch_users():
12     try:
13         # Send GET Request to the API
14         response = requests.get(API_URL, timeout=10)
15
16         # Check for HTTP errors
17         response.raise_for_status()
18
19         # Convert JSON response into Python data
20         users = response.json()
21
22         print("\nAPI data fetched successfully!")
23         print("Total users:", len(users))
24
25         return users
26
27     except requests.exceptions.Timeout:
28         print("\nError: API request timed out.")
29
```

Output Debug Console Problems Terminal Ports Spell Checker

PS C:\Users\RAKESH\OneDrive\ドキュメント\Desktop\API_JSON_Project> python api_user_manager.py

Enter your choice: 3

Enter city to filter: Gwenborough

Users from Gwenborough :

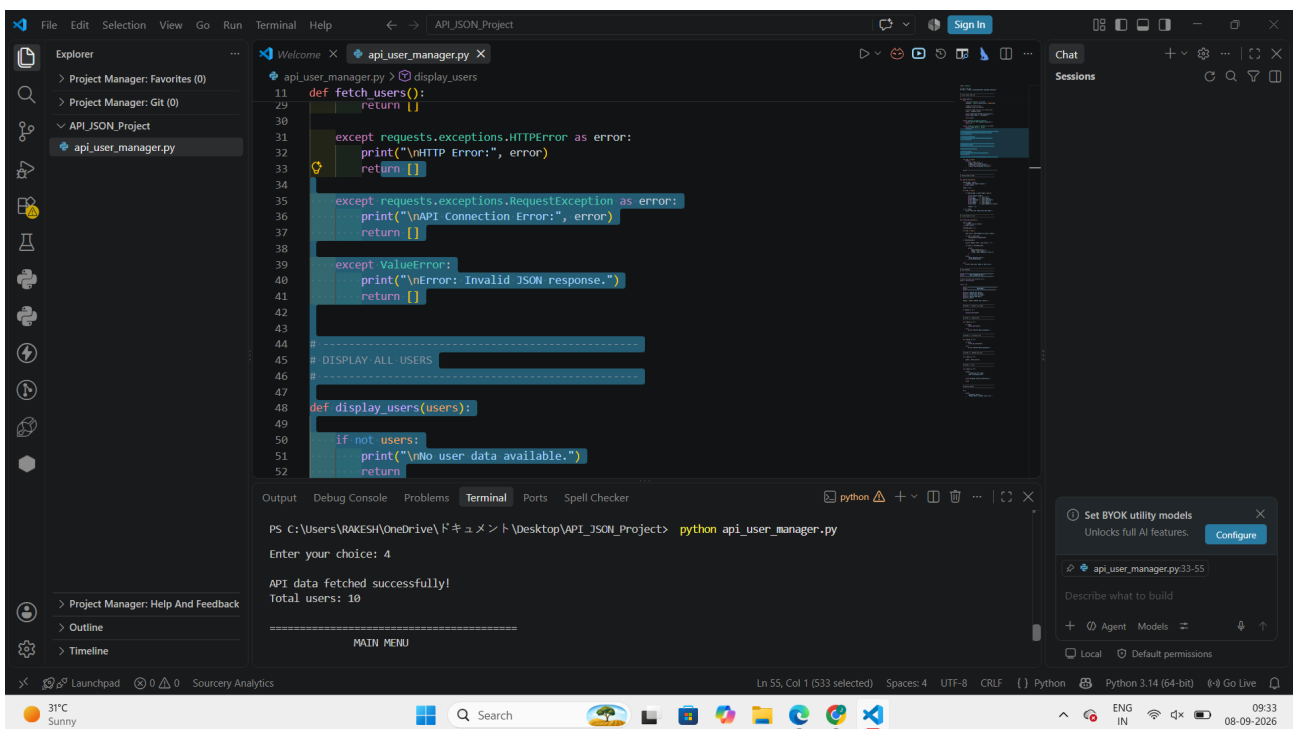
ID: 1 | Name: Leanne Graham | City: Gwenborough

Total matching users: 1

=====

The program filters nested address-city data and displays the matching user records.

4. Refresh API Data



The screenshot shows the VS Code editor with the file `api_user_manager.py` open. The code defines a `fetch_users()` function that sends a GET request to an API, checks for errors, and returns the JSON response. The terminal output shows the command `python api_user_manager.py` being executed, followed by the prompt `Enter your choice: 4` and `API data fetched successfully!` and `Total users: 10`.

```
11 def fetch_users():
29     return []
30
31 except requests.exceptions.HTTPError as error:
32     print("\nHTTP Error:", error)
33     return []
34
35 except requests.exceptions.RequestException as error:
36     print("\nAPI Connection Error:", error)
37     return []
38
39 except ValueError:
40     print("\nError: Invalid JSON response.")
41     return []
42
43
44
45 # DISPLAY ALL USERS
46 # -----
47
48 def display_users(users):
49
50     if not users:
51         print("\nNo user data available.")
52         return
53
```

Output Debug Console Problems Terminal Ports Spell Checker

PS C:\Users\RAKESH\OneDrive\ドキュメント\Desktop\API_JSON_Project> python api_user_manager.py

Enter your choice: 4

API data fetched successfully!

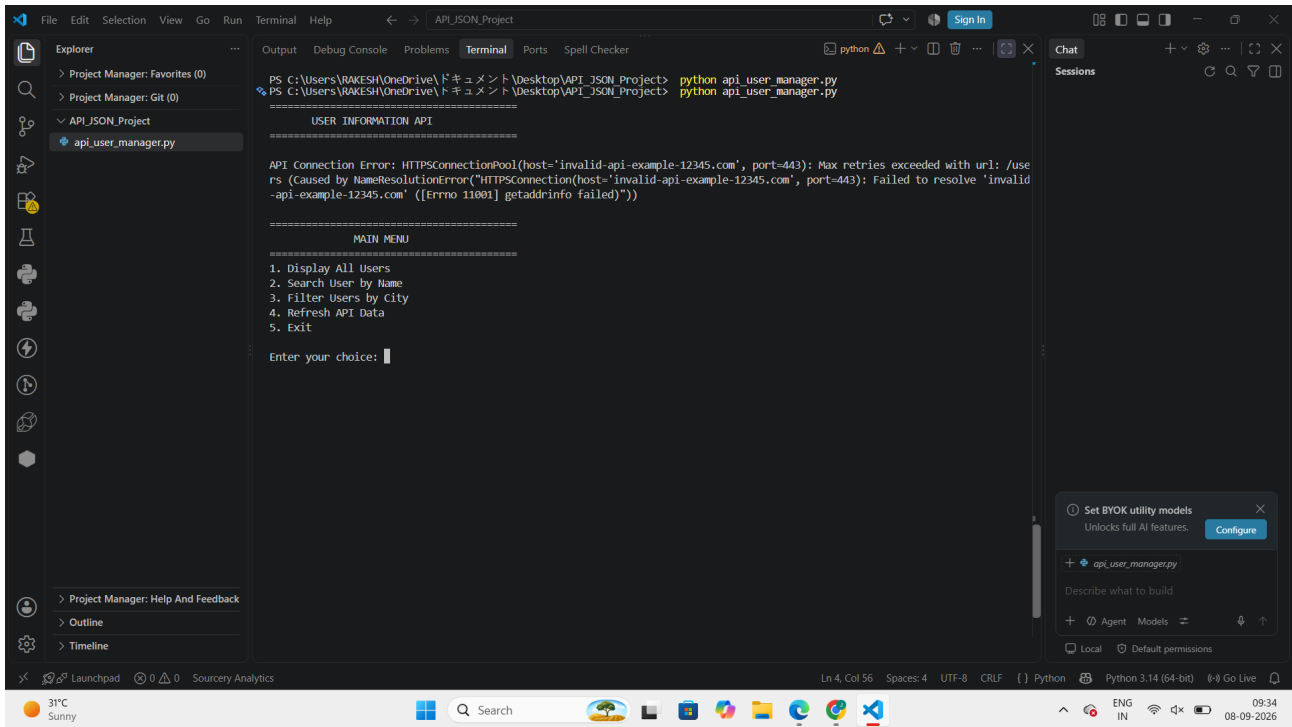
Total users: 10

=====

MAIN MENU

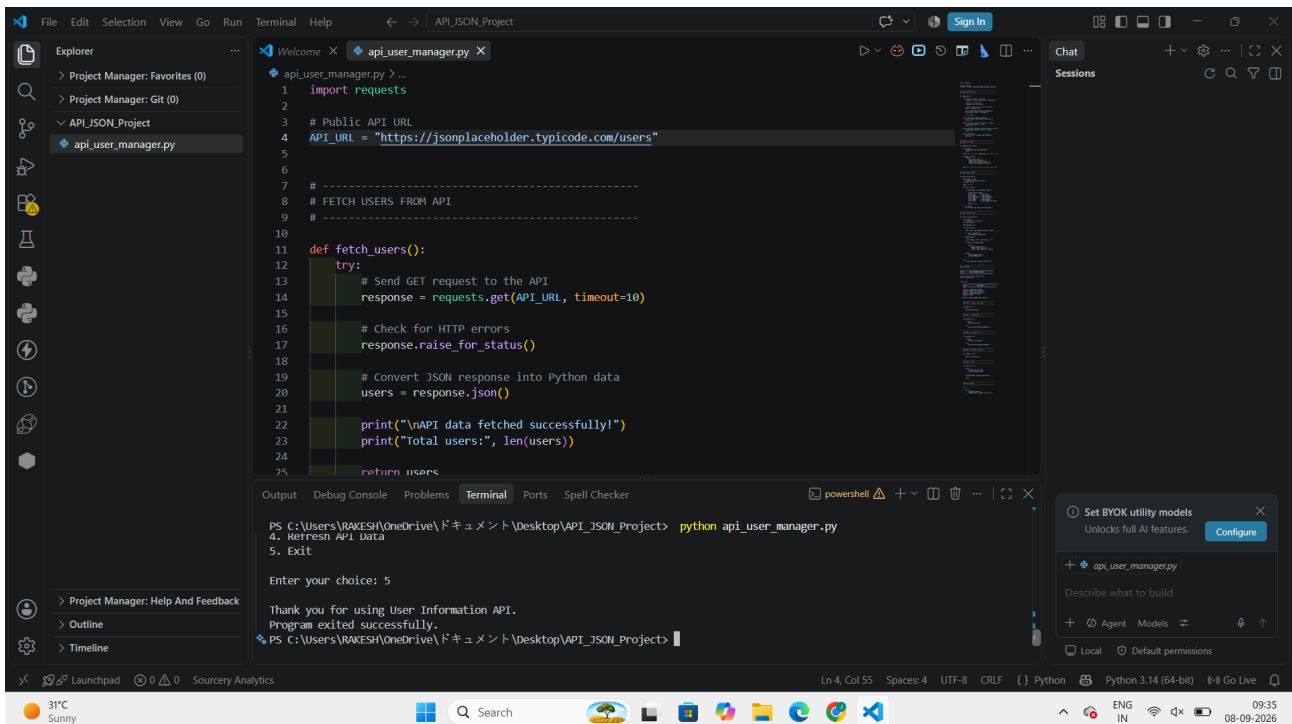
The refresh option performs another API request and successfully retrieves the JSON data.

5. API Error Handling



An invalid API address was used temporarily. The program caught the connection error and continued to the menu instead of crashing.

6. Program Exit



The application exits cleanly after option 5. The correct working API URL was restored before final use.

12. Advantages

- Simple menu-driven interface
- Real API communication
- Structured JSON processing
- Practical search and filtering

- Robust exception handling
- Easy to extend

13. Conclusion

The project successfully demonstrates API integration and JSON handling in Python. It retrieves external data with Requests, parses JSON, supports search and city filtering, refreshes data, and handles API failures without crashing.

14. Submission Link

<https://github.com/bammidirakesh/Python-File-Automation>